

IMY 210 Project

XML, XSLT, Vue & Docker Integration

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Ensure that all worked-on files have your surname and student number (Grunewald, u12345678) in them.

Project Scenario

You are developing a document transformation platform that allows users to upload structured data and presentation templates to automatically generate web pages.

The system works as follows:

The system functions similarly to a dynamic template engine, where structured data (XML) is combined with presentation templates (XSLT) to automatically generate web pages.

- XML files store structured content (e.g. schedules, directories, FAQs).
- XSD files define and validate the required XML structure.
- XSLT files define how that content should be presented.
- The server processes and transforms the XML using the selected XSLT template.
- The output is generated as HTML (and optionally PDF). HTML output is required. PDF generation is optional and may be implemented as an extension using XSL-FO.
- A Vue-based frontend allows users to upload files, trigger transformations, and view the results.

This project simulates a real-world system where:

- Structured data is separated from presentation
- Users can apply different templates to the same data
- legacy XML workflows are integrated into a modern web application

Project Theme

You will build a small **information portal** (for example, a campus or student information hub).

You may choose your own theme, but it must include structured content that makes sense to store in XML, such as:

- Schedules or timetables
- Campus services/offices directory
- FAQs/help pages
- Staff or venue listings

This is an individual project completed incrementally across the semester.

What You Will Build (Final System)

You will build a web-based application for transforming XML data into web pages.

The system will allow a user to:

- Upload XML data files
- Upload XSLT templates
- Select which XML + XSLT combination to use
- Trigger a transformation on the backend
- View the generated HTML output in the browser

The final system will demonstrate how structured data can be transformed into web pages using a backend transformation pipeline and a Vue interface.

Example User Flow

A user visits the application and:

1. Uploads an XML file containing structured information
2. Uploads an XSLT template that defines how the information should appear
3. Selects which XML file and template to use
4. Clicks a “Transform” button
5. The backend processes the files and generates an HTML page
6. The generated page is displayed in the browser

How the System Works

This project simulates a real-world document transformation platform where structured data is separated from presentation.

The system consists of four main layers:

1) Structured Data Layer

- XML files store structured content
- XSD schemas validate the structure of XML
- XML content used in the system must conform to its associated XSD schema that defines its valid structure.
- The backend must validate the XML against its XSD before performing any XSLT transformation.
- The XML, XSD, and XSLT files created in Mini Hand-in 1 form the structured transformation pipeline that will be reused and extended throughout the rest of the project.
- XSLT stylesheets define how XML content is presented

These represent the data and presentation components of the system.

2) Backend Layer

A RESTful backend server manages and processes structured data.

The backend must:

- Accept XML uploads
- Accept XSLT template uploads
- Store and manage files
- Perform CRUD operations
- Validate XML using XSD
- Execute XSLT transformations
- Return generated output (HTML and/or PDF)

The transformation process must occur on the server.

3) Frontend Layer (Vue)

Vue functions as the user interface for the system.

The Vue application must:

- Allow users to upload XML and XSLT files
- Allow users to select XML + XSLT combinations
- Trigger transformations via the backend
- Fetch generated output
- Display transformed HTML in the browser

Vue does not perform XML transformations. It only sends requests to the backend and displays the generated results.

4) Environment & Deployment

Deployment tooling such as Docker may be used to create a reproducible environment.

Docker is optional and will be treated as a bonus component.

Conceptual Model

- XML represents the content/data
- XSLT represents the presentation template
- XSD defines the rules that XML must follow
- The backend performs the transformation
- Vue acts as the user interface to trigger and view results

Project Overview

In this project, you will build a document transformation platform that integrates:

- XML structured data
- XSD validation
- XSLT transformations
- a RESTful backend
- a Vue frontend

Your final application must demonstrate a working transformation pipeline:

XML + XSLT → backend processing → HTML output → Vue display

This project is assessed as a full-stack system, requiring integration between structured data, backend processing, and a Vue frontend.

Project Development Phases

This project is completed in three progressive mini hand-ins.

Each phase builds on the previous one and introduces a new system component.

The goal is to develop the project incrementally rather than building everything at once.

Phase Overview

Mini Hand-in 1 — Data and Transformation Foundation

- Build structured XML content
- Validate XML using XSD
- Create XSLT templates
- Establish a working transformation pipeline on the backend

Mini Hand-in 2 — Backend System

- Implement a RESTful backend
- Support CRUD operations for XML and XSLT
- Execute transformations via backend endpoints
- Connect backend functionality to the transformation pipeline
- A basic Vue interface may be used for testing

Mini Hand-in 3 — Frontend and Integration

- Build the Vue application
- Connect Vue to backend services
- Trigger transformations from the UI
- Display generated results
- Demonstrate full system integration

Each mini hand-in represents a functional milestone of the final system.

Mini Hand-in 1: XML Data + XSD Validation + XSLT Transformation Pipeline

Purpose

This mini hand-in focuses on establishing the structured data and transformation foundation of the project.

You must create the XML content, enforce its structure using XSD, and implement an XSLT transformation process executed by a backend server.

This stage establishes the pipeline used throughout the rest of the project:

XML + XSLT → Server transformation → HTML output

Requirements

1. XML Content

Create one or more XML files containing meaningful structured data for your chosen theme.

The XML must:

- Contain realistic, meaningful content (not placeholder text)
- Use a clear structure with appropriate elements and attributes
- Represent information that could realistically be transformed into a webpage

Examples include:

- Schedules or timetables,
- Service directories,
- FAQs/help pages,
- Staff or venue listings,
- Information portals

2. XSD Validation

Create an XSD schema that defines and validates the structure of your XML.

The schema must enforce:

- Correct element hierarchy
- Required elements and/or attributes
- Appropriate data types (strings, numbers, dates, etc.)

You must provide evidence that your XML has been validated against the XSD.

Evidence may include:

- Validation output
- Screenshots
- Logs
- Explanation in README

3. XSLT Transformation

Create XSLT stylesheets that define how the XML content is transformed into HTML.

The transformation must:

- Render readable, structured HTML
- Use `xsl:for-each` to display repeated data
- Use conditional logic (`xsl:if` or `xsl:choose`)

4) Server-Side Transformation Setup

A minimal backend capable of executing transformations is sufficient. Full REST functionality will be implemented in Mini Hand-in 2.

At this stage, the server must:

- Accept XML files
- Accept XSLT templates
- Execute the transformation process
- Return generated HTML output

The transformation must occur on the server, not in Vue.

Appropriate transformation engines may be used, such as:

- XSLT processors (e.g. libxslt)
- XSL-FO processors for document generation (e.g. Apache FOP)

Processor note (cross-platform compatibility)

You may use any suitable server-side XSLT processor. Some Node-based libraries require native compilation and may not work consistently across different operating systems.

For a stable cross-platform option, you may optionally use a Java-based XSLT processor such as Saxon-HE (Home Edition) (Java required). Depending on the Saxon version, the XML Resolver library may also be required on the classpath.

You may choose an alternative processor if you prefer, provided that:

- The transformation occurs on the server
- The XML is transformed using XSLT
- The system produces HTML output

Mini Hand-in 1 Submission Requirements

You must submit:

- XML file(s) containing project content
- XSD schema file(s)
- XSLT stylesheet file(s)
- backend server setup capable of running transformations
- evidence of XML validation
- evidence that a transformation pipeline runs successfully

Mini Hand-in 1 Submission Format and Packaging

- Compress all project files, excluding the `node_modules` folder, into a single ZIP archive.
- Name the ZIP file exactly as follows: `project_mini1_uXXXXXXXXX.zip` (where `uXXXXXXXXX` is your student number, e.g. `project_mini1_u12345678.zip`)
- Submit the ZIP file to the upload slot provided on ClickUP before the deadline.
- **No late submissions will be accepted.**
- Ensure that you keep a backup copy of your submission.

Incorrect file naming, missing files, or inclusion of the `node_modules` folder may result in penalties.

Mini Hand-in 2: Backend Implementation + CRUD + Vue Integration

Purpose

This mini hand-in focuses on building the operational backend system. Integration with Vue will occur in Mini Hand-in 3.

You must implement a RESTful backend capable of managing the XML and XSLT files created in Mini Hand-in 1 and triggering the transformation process.

Backend functionality is the primary focus of this phase.

Endpoints should be tested using Postman (or an equivalent API testing tool).

If testing file upload endpoints, you may use Postman form-data.

A full Vue frontend is not required at this stage. A minimal interface may be used only if needed for testing.

Scope of this Phase

This phase focuses on backend implementation and server functionality.

The Vue interface does not need to be fully developed yet. Simple pages or components used to test backend requests are sufficient.

The primary goal is to ensure that:

- Backend endpoints work correctly
- XML/XSLT files are stored and managed
- Transformations execute successfully

The Vue interface may be minimal and does not need styling polish. This phase is assessed primarily on backend functionality.

Requirements

1) Backend Implementation

You must build a RESTful backend server that:

- Accepts XML file uploads
- Accepts XSLT template uploads
- Stores and manages uploaded files
- Performs CRUD operations on XML and XSLT resources
- Validates XML using XSD

- Executes XSLT transformations using a server-side XSLT processor (e.g. Saxon-HE, libxslt, or equivalent).
- Returns generated output (HTML and/or PDF)

The backend must use the XML, XSD, and XSLT files created during Mini Hand-in 1.

2) *CRUD Operations*

The backend must support basic CRUD functionality:

- Create → upload XML/XSLT
- Read → retrieve stored files
- Update → replace or modify files
- Delete → remove files

3) *Frontend (Optional for Testing)*

A minimal Vue interface may be created to assist with testing backend functionality.

A full frontend implementation is not required in this phase and will be assessed in Mini Hand-in 3.

Mini Hand-in 2 Submission Requirements

You must submit:

- Backend source code
- Vue project source files (only if used for testing)
- XML/XSD/XSLT files from Mini Hand-in 1
- *README.md* containing:
 - o Project description
 - o Instructions to run backend
 - o Instructions to run Vue frontend
 - o Explanation of CRUD endpoints
 - o Explanation of transformation workflow

Mini Hand-in 2 Submission Format and Packaging

- Compress all project files, excluding the `node_modules` folder, into a single ZIP archive.

- Name the ZIP file exactly as follows: `project_mini2_uXXXXXXXXX.zip`
(where `uXXXXXXXXX` is your student number, e.g.
`project_mini2_u12345678.zip`)
- Submit the ZIP file to the upload slot provided on ClickUP before the deadline.
- **No late submissions will be accepted.**
- Ensure that you keep a backup copy of your submission.

Incorrect file naming, missing files, or inclusion of the `node_modules` folder may result in penalties.

Mini Hand-in 3: Frontend Implementation + System Integration

Purpose

This mini hand-in focuses on building the frontend system and integrating all previously developed components into a complete working application.

The backend and transformation pipeline developed in Mini Hand-ins 1 and 2 must now be accessed through a Vue interface.

You must implement a Vue-based interface that communicates with the backend and allows users to interact with the XML transformation system.

Scope of this Phase

By this stage:

- The backend must already be functional
- XML/XSD/XSLT pipeline must already work
- Transformations must already run on the server

The goal of this phase is to:

- Build a usable interface
- Connect Vue to backend services
- Demonstrate the complete end-to-end system

Requirements

The interface should guide a user through uploading data, selecting templates, triggering a transformation, and viewing the result.

1) Vue Application

The application must:

- Allow users to upload XML files
- Allow users to upload XSLT templates
- Allow users to select XML + XSLT combinations
- Trigger transformations via backend requests
- Fetch generated output
- Display transformed HTML in the browser (required)
- Optionally support PDF viewing or download if implemented in the backend

Vue must not perform the transformation itself.

2) Integration

The system must demonstrate full integration between:

- XML data
- XSD validation
- XSLT transformation
- backend processing
- Vue interface

The transformation pipeline must function end-to-end.

3) Routing & Interface Expectations

- Vue Router must be used
- Multiple pages/views must exist
- Navigation must not reload the browser
- Basic loading and error states must be handled

Mini Hand-in 3 Submission Requirements

You must submit:

- Vue project source files
- backend source files
- XML/XSD/XSLT files
- *README.md* containing:
 - o Project description
 - o System architecture overview
 - o Instructions to run the backend
 - o Instructions to run the frontend
 - o Explanation of the transformation flow

Mini Hand-in 3 Submission Format and Packaging

- Compress all project files, excluding the `node_modules` folder, into a single ZIP archive.
- Name the ZIP file exactly as follows: `project_mini3_uXXXXXXXXX.zip`
(where `uXXXXXXXXX` is your student number, e.g.
`project_mini3_u12345678.zip`)

- Submit the ZIP file to the upload slot provided on ClickUP before the deadline.
- **No late submissions will be accepted.**
- Ensure that you keep a backup copy of your submission.

Incorrect file naming, missing files, or inclusion of the `node_modules` folder may result in penalties.

Bonus Component: Docker & Deployment

Docker is not a required submission for the mini hand-ins, but may be included as an extension or bonus component.

Students who choose to implement Docker should aim to:

- Containerise the backend server
- Containerise the Vue frontend
- Ensure the transformation pipeline runs inside the container
- Provide instructions to build and run the system

Final Submission Requirements

The final submission must demonstrate a fully working document transformation system integrating all project components.

Your final project must include:

Core System

- Backend source code
- Vue frontend source code
- XML content files with meaningful structured data
- XSD schema validation of XML (with evidence)
- XSLT templates generating HTML output
- REST endpoints supporting CRUD operations for XML and XSLT files
- Full end-to-end transformation pipeline:

XML + XSLT → backend transformation → HTML output → Vue display

Documentation

A complete `README.md` containing:

- Project description
- System architecture overview
- Instructions to run the backend
- Instructions to run the frontend
- Explanation of the transformation workflow
- Explanation of how XML validation was performed

Optional Bonus (Docker)

If Docker is included, provide:

- Container setup for backend and/or frontend
- Instructions to build and run
- Confirmation that the transformation pipeline runs in the container

Submission Format and Packaging

- Compress all project files, excluding the `node_modules` folder, into a single ZIP archive.
- Name the ZIP file exactly as follows: `project_uXXXXXXXX.zip` (where `uXXXXXXXX` is your student number, e.g. `project_u12345678.zip`)
- Submit the ZIP file to the upload slot provided on ClickUP before the deadline.
- **No late submissions will be accepted.**
- Ensure that you keep a backup copy of your final submission.

Incorrect file naming, missing files, or inclusion of the `node_modules` folder may result in penalties.

Rules and Constraints

- Vue must be used as the frontend framework.
- XML must be transformed using XSLT (not manually rewritten as HTML).
- XML must be validated using XSD.
- Docker is optional and will be treated as a bonus component.
- Native browser or platform functionality must be used where applicable.
- Code must be readable, structured, and appropriately commented.

Assessment Notes

Marks will be awarded based on:

- Correct implementation of XML, XSD, and XSLT
- Successful integration with Vue

- Correct SPA routing behaviour
- Docker implementation (bonus), where applicable
- Clarity, structure, and correctness of the submission

Incomplete or non-functional submissions will receive reduced marks.

Final Notes

This project is intended to demonstrate your ability to:

- Work with structured data formats
- Transform and validate XML
- Integrate legacy data formats into modern web frameworks
- Set up reproducible development environments
- Plan your project carefully and test each component before submission.